

Disclaimer: Dieses OER-Dokument stammt aus dem Projekt [proTirone](#), das über GitHub [freie Unterrichtsmaterialien](#) samt [Quellcode](#) offeriert. proTirone-Materialien werden unter den Bedingungen der [CC-BY-4.0-Lizenz](#) so angeboten, wie sie sind, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).

LF09:15:Sockets & Ports (& Services)

ÜBUNG LF09/15:services:01

- 1) Bitte wählen Sie
 - 2 Mitschülerinnen, die je eine Routerhälfte mit eingebettetem Switch simulieren
 - 2 Mitschülerinnen, die je ein Betriebssystem simulieren
 - 2 Mitschülerinnen, die je eine Netzwerkkarte simulieren
 - 2 Mitschülerinnen, die je eine App simulieren
- 2) Wählen Sie ein oder zwei Protokollantinnen
- 3) (Er)finden Sie einen allgemeingültigen, mehrfach parallel ausführbaren Prozess,
 - der die bisher besprochenen Techniken nutzt,
 - der damit eine Nachricht von APP(BCA) zu APP(BCB) und die Antwort zurück übermittelt und
 - der die Regel beachtet, dass nur das Betriebssystem von der Netzwerkkarte lesen und auf die Netzwerkkarte schreiben kann.
- 4) Führen Sie Ihren Prozess als Schreibtischtest durch.
- 5) Dokumentieren Sie Ihren Prozess als Aktivitätsdiagramm.

ÜBUNG LF09/15:services:02

- ☐ Bitte teilen Sie sich in vier Gruppen auf.
- ☐ Bitte nehmen Sie gruppenweise Ihr ‘Geschenk’ entgegen und entpacken Sie es vorsichtig.
- ☐ Kriegen Sie bitte heraus, was Sie da bekommen haben. (ca. 40 Min).
- ☐ Erstellen Sie eine kurze Präsentation, anhand derer Sie Ihren Mitschülerinnen charmant erläutern können, was Sie da bekommen haben (ca. 20 Min.). Leitfragen dazu sind:
 - Was haben Sie bekommen?

- Wie ist es aufgebaut und wie funktioniert es?
 - Wofür stehen die Komponenten des 'Geschenks' in der Wirklichkeit?
 - Präsentieren Sie Ihr Ergebnis (Pkg 1, 2, 3, 4)
-

ÜBUNG LF09/15:services:03

- ☐ Lassen Sie sich über die folgenden Befehle die auf Ihrem Rechner arbeitenden Server-Applikationen und ihre Ports anzeigen:
 - LNX/MACOS: `netstat -ano | grep tcp`
 - LNX/MACOS: `netstat -ano | grep udp`
 - LNX/MACOS: `lsof -i -n | grep tcp`
 - W11/pwsh (V.7.x): `netstat -ano | select-string TCP`
 - W11/powershell (V.5.1): `netstat -ano | findstr HERGESTELLT`
 - ☐ Führen Sie diese Befehle nun als Root/Admin aus (LNX: `sudo lsof -i -n`). Beschreiben Sie, was jetzt angezeigt wird.?
 - ☐ Beschreiben Sie, was auf Ihrem Rechner 'los ist'.
-

ÜBUNG LF09/15:services:04

- ☐ Programmieren Sie in Python einen TCP/IP-Echo-Server (mit Pythonmodul `socket`), der
 - ☐ auf der Konsole mit `python3 echo-server.py` gestartet wird,
 - ☐ hintereinander mehrere Requests an die Loopbackadresse entgegennimmt,
 - ☐ sich für einen festen Port aus dem Bereich der dynamischen Ports zuständig fühlt,
 - ☐ anzeigt, wann er zu beantworten bereit ist,
 - ☐ die Daten des anfragenden Clients anzeigt,
 - ☐ den Erfolg einer Rückmeldung anzeigt.
- ☐ Programmieren Sie in Python einen Echo-Client (mit Pythonmodul `socket`), der
 - ☐ auf der Konsole mit `python3 echo-client.py` gestartet wird,
 - ☐ nach Erhalt des Sockets konfigurierbar lange abwartet, bevor er
 - ☐ einen Echo-Request an die Loopbackadresse absetzt,
 - ☐ dabei den oben festgelegten Port als Zielpport benutzt,
 - ☐ den Request vor Absenden ausgibt,

- ☐ sein Warten auf die Antwort signalisiert,
 - ☐ die eingegangene Antwort ausgibt.
 - ☐ Testen Sie den Service über zwei Konsolenfenster.
 - → <https://docs.python.org/3/library/socket.html>
 - → https://www.w3schools.com/python/ref_module_socket.asp
-

ÜBUNG LF09/15:services:05

- ☐ Öffnen Sie 3 Terminals nebeneinander.
 - ☐ Starten Sie im ersten Terminal Ihren Echo-Server mit `python3 echo-server.py`.
 - ☐ Starten Sie im zweiten Terminal Ihren Echo-Server mit `python3 echoclient.py`.
 - ☐ Rufen Sie im dritten Terminal (fortlaufend) vor jeder Ausgabe in Terminal 2 `lsof -i -n`.
 - ☐ Beschreiben Sie, wie sich der Status Ihres Echo-Servers immer wieder verändert.
-